<<componente>> Torneos

<<Controller>>
GestorTorneos

torneos: Map<String, Torneo>
consecutivo: int
topK: int

crearTorneo(datos...): Torneo
buscar(id: String): Torneo
listar(): List<Torneo>
esCupoPreferencial(p: ParticipanteTorneo, t: Torneo): boolean
empleadoLibre(e: Empleado, fecha: LocalDate): boolean

<<Information holder>>
Ranking (P1)

<<Controller>>
RegistroTurnos (P1)

<<Information holder>>
Torneo {abstract}

id: String
nombre: String
disciplina: String
modalidad: Modalidad
fecha: LocalDate
categoria: Categoria
cuposTotales: int
cuposPreferenciales: int
estado: EstadoTorneo
premiosEntregados: boolean

calcularCuposPreferenciales(): int
cuposRegularesLibres(): int
cuposPreferencialesLibres(): int
inscribir(inscripcion: InscripcionTorneo)
desinscribir(login: String)
iniciar(hoy: LocalDate)
registrarResultado(idEncuentro: String, local: int, visitante: int)
finalizar()
permiteEmpates(): boolean
{abstract} calcularCobro(p: ParticipanteTorneo): double
{abstract} repartirPremios(tabla: List<PosicionTorneo>)

<<Information holder>>
Categoria

edadMaxima: Integer
nivel: Nivel

admite(edad: int, nivel: Nivel): boolean

<<enumeration>>
Nivel

PRINCIPIANTE
INTERMEDIO
AVANZADO

<<enumeration>>
Modalidad

CONJUNTO
INDIVIDUAL
DOBLES

<<enumeration>>
EstadoTorneo

INSCRIPCIONES_ABIERTAS
EN_CURSO
FINALIZADO

Convenciones
Amarillo clase nueva del Proyecto 2
Azul clase del P1 modificada
Gris punteado clase del P1 sin cambios

<<Information holder>>
InscripcionTorneo

participante: ParticipanteTorneo
preferencial: boolean
montoPagado: double
confirmacionPago: String
orden: int

cuposQueOcupa(): int

<<Information holder>>
TorneoRecreativo

porcentajeBono: double = 20

calcularCobro(p: ParticipanteTorneo): double
repartirPremios(tabla: List<PosicionTorneo>)

<<Information holder>>
TorneoCompetitivo

tarifa: double
fondoPremios: double

calcularCobro(p: ParticipanteTorneo): double
repartirPremios(tabla: List<PosicionTorneo>)
devolverTarifa(inscripcion: InscripcionTorneo)

<<Structurer>>
TablaTorneo

posiciones: List<PosicionTorneo>

actualizar(encuentro: Encuentro, permiteEmpate: boolean)
ordenarConDesempate(): List<PosicionTorneo>

<<Information holder>>
PosicionTorneo

participante: ParticipanteTorneo
puntos: int
marcadorFavor: int
marcadorContra: int
ordenInscripcion: int

diferencia(): int

<<Information holder>>
Encuentro

id: String
ronda: int
local: ParticipanteTorneo
visitante: ParticipanteTorneo
marcadorLocal: int
marcadorVisitante: int
jugado: boolean

registrarResultado(local: int, visitante: int)
esDescanso(): boolean
ganador(): ParticipanteTorneo

Reglas que viven aquí (HU-T01 a T13)
• 20% de cupos preferenciales, redondeado hacia arriba (en dobles, sobre parejas).
• Recreativo: sin tarifa; premia con bono del 20%.
• Competitivo: tarifa > 0; fondo 60-30-10; empleados no pagan ni reciben efectivo.
• Liga todos contra todos; puntos 3-1-0; empates solo en conjunto si el deporte lo permite.

<<interface>>
ParticipanteTorneo

darNombre(): String
darIntegrantes(): List<Usuario>
promedioElo(disciplina: String): double
incluyeEmpleado(): boolean
contiene(login: String): boolean

<<Information holder>>
Jugador

usuario: Usuario
...métodos de la interfaz

<<Information holder>>
Pareja

jugador1: Usuario
jugador2: Usuario
...métodos de la interfaz

<<Structurer>>
EquipoTorneo

nombre: String
capitan: Socio
integrantes: List<Socio>

esCapitan(login: String): boolean
...métodos de la interfaz

<<Information holder>>
Usuario (modificada)

<<Information holder>>
Socio (modificada)

<<Information holder>>
Premio {abstract}

puesto: int
beneficiario: ParticipanteTorneo

describir(): String

<<Information holder>>
PremioEfectivo

monto: double

describir(): String

<<Information holder>>
BonoDescuento

id: String
porcentaje: double
titular: Usuario
usado: boolean

canjear(login: String, subtotal: double, otroDescuento: boolean): double
describir(): String

<<Service provider>>
GeneradorCalendario

generarLiga(participantes: List<ParticipanteTorneo>): List<Encuentro>

crea